```
  if (options.allowOrigin) {
    var origin = req.headers['origin'];
    if (options.allowOrigin(origin)) {
      res.set('Access-Control-Allow-Origin', origin);
    }
  } else {
    res.set('Access-Control-Allow-Origin', '*');
  }
  …
};
```

Checks if allow-Origin option exists

Runs allowOrigin function against origin header

Sets Access-Control-Allow-Origin header if origin is valid

If allowOrigin option isn't specified, allows all origins

The code starts by creating an array of valid client origins. Because the sample code accepts cross-origin requests only from http://localhost:1111, this is the only value in the whitelist. The code then defines a new `allowOrigin` option which uses the `createWhitelistValidator` function to create a function to validate the origin.

Now that you've created the `allowOrigin` option, you need to read that value from your middleware. The code starts by checking if the `allowOrigin` option exists. If it doesn't, the code sets the Access-Control-Allow-Origin header to `*`, as it did originally. But if the `allowOrigin` does exist, the code runs the validator function against the value of the origin header. If there is a match, the origin is allowed to make cross-origin requests, and you can set the Access-Control-Allow-Origin header to the value of the Origin header. Otherwise, the client isn't allowed to make cross-origin requests, and the Access-Control-Allow-Origin header is never set.

A whitelist is an easy way to manage cross-origin access to your server. You can easily add values to the `originWhitelist` array to give access to more servers. But it also requires constant management to make sure it's up-to-date.

OTHER WAYS TO VALIDATE THE ORIGIN

A whitelist is not the only way you can validate client origin values. You can also modify listing 6.3 to use a regular expression or issue a database call to validate the origin. Suppose instead of just three clients you want to allow any client from http://localhost, regardless of port, to access your API. You could use a regular expression to validate this origin, as follows:

```
var createRegexpValidator = function(re) {
  return function(origin) {
    return re.test(origin);
  }
};

var corsOptions = {
  allowOrigin: createRegexpValidator(
      /^http:\/\/localhost(:\d+)?$/i)
};
```

createRegexpValidator creates a function to validate an origin using a regular expression.

This regular expression matches a request to any port on http://localhost.

This code snippet introduces a new `createRegexpValidator` function. The `createRegexpValidator` function works in the same way as the `createWhitelistValidator` function from listing 6.2, except instead of a whitelist check, this function runs a regular expression. Any client of the form http://localhost:PORT (where PORT is the port